

Document Chunk API

Visual RAG Document Assistant — REST API Reference

Visual RAG Team

February 15, 2026

Abstract

This document describes the Document Chunk API, a standalone FastAPI backend that provides REST endpoints for uploading, searching, and managing document chunks. The API enables external frontends, other backends, or any HTTP client to interact with the Visual RAG system independently of the Chainlit chat UI.

Contents

1	Overview	3
1.1	Supported File Types	3
2	Authentication	3
3	Endpoints	3
3.1	Upload Document	3
3.1.1	Request Parameters	4
3.1.2	Success Response — 200 OK	4
3.1.3	Error Responses	4
3.1.4	Examples	4
3.2	Get Chunks	6
3.2.1	Query Parameters	6
3.2.2	Success Response — 200 OK	6
3.2.3	Chunk Object Fields	7
3.2.4	Error Responses	7
3.2.5	Examples	7
3.3	Delete Document	8
3.3.1	Path Parameters	8
3.3.2	Success Response — 200 OK	8
3.3.3	Error Responses	8
3.3.4	What Happens on Delete	8
3.3.5	Examples	8
3.4	Health Check	9
3.4.1	Success Response — 200 OK	9
3.4.2	Example	9
4	Complete Workflow Example	10
4.1	Terminal (cURL)	10
4.2	Shell Script (Automated Workflow)	10
5	Frontend Integration	11
5.1	JavaScript (Fetch API)	11
5.2	Python (requests)	11

6	Configuration	13
6.1	Running Locally (Without Docker)	13
6.2	Running with Docker	13
7	Response Schema Reference	13
7.1	UploadResponse	13
7.2	ChunksResponse	13
7.3	DeleteResponse	14
7.4	Error Response (HTTPException)	14
8	Notes	14

1 Overview

The Document Chunk API exposes document management functionality over standard HTTP/REST. It runs alongside the Chainlit web UI but operates independently with its own document index.

Property	Value
Framework	FastAPI
Default Port	8010 (configurable via API_PORT in .env)
Base URL	http://localhost:8010
Swagger Docs	http://localhost:8010/docs
ReDoc	http://localhost:8010/redoc
CORS	All origins allowed (configure for production)
Content Type	multipart/form-data (upload), application/json (responses)

Table 1: API overview

1.1 Supported File Types

The API accepts the following file formats for upload:

- **PDF** — .pdf
- **Images** — .png, .jpg, .jpeg, .webp
- **Word Documents** — .docx
- **Text Files** — .txt
- **JSON Files** — .json

Non-PDF/image files are automatically converted to PDF before indexing.

2 Authentication

The API currently has **no authentication**. All endpoints are publicly accessible. For production deployments, consider adding API key or OAuth2 authentication via FastAPI's built-in security utilities.

3 Endpoints

3.1 Upload Document

Method	POST
Path	/api/documents/upload
Content-Type	multipart/form-data

Upload a document to the system. The server splits the document into chunks (one per page) and indexes them for visual similarity search.

3.1.1 Request Parameters

Field	Type	Required	In	Description
file	file	Yes	body	The document file to upload
document_id	string	No	body	Custom document ID (auto-generated UUID if omitted)
document_name	string	No	body	Display name (defaults to the original file-name)

Table 2: Upload request parameters

3.1.2 Success Response — 200 OK

```
{
  "document_id": "550e8400-e29b-41d4-a716-446655440000",
  "document_name": "report.pdf",
  "chunk_count": 12,
  "message": "Document uploaded and split into 12 chunk(s)."
}
```

Field	Type	Description
document_id	string	Unique identifier for the uploaded document
document_name	string	Display name of the document
chunk_count	integer	Number of chunks (pages) the document was split into
message	string	Human-readable status message

Table 3: Upload response fields

3.1.3 Error Responses

Status Code	Description
400 Bad Request	Unsupported file type. The response body includes the list of supported extensions.
409 Conflict	A document with the given <code>document_id</code> already exists. Delete it first or use a different ID.
500 Internal Server Error	Document processing failed (conversion, indexing, or storage error).

Table 4: Upload error responses

3.1.4 Examples

cURL — Upload a PDF:

```
curl -X POST http://localhost:8010/api/documents/upload \
  -F "file=@report.pdf" \
  -F "document_id=my-doc-001" \
  -F "document_name=Annual Report"
```

cURL — Upload an image (auto-generated ID):

```
curl -X POST http://localhost:8010/api/documents/upload \  
-F "file=@screenshot.png"
```

cURL — Upload a Word document:

```
curl -X POST http://localhost:8010/api/documents/upload \  
-F "file=@meeting_notes.docx" \  
-F "document_name=Meeting Notes Q1"
```

3.2 Get Chunks

Method	GET
Path	/api/documents/chunks
Content-Type	application/json (response)

Retrieve document chunks that are most relevant to a natural language query. The system uses ColQwen2 visual embeddings to find the most similar pages.

3.2.1 Query Parameters

Parameter	Type	Required	Default	Range	Description
query	string	Yes	—	—	Natural language query to search for
document_id	string	No	—	—	Filter results to a specific document
top_k	integer	No	5	1–50	Number of chunks to return

Table 5: Get chunks query parameters

3.2.2 Success Response — 200 OK

```
{
  "query": "What is the total revenue?",
  "chunks": [
    {
      "document_id": "my-doc-001",
      "document_name": "Annual Report",
      "chunk_index": 4,
      "score": 0.847,
      "image_base64": "iVBORwOKGgo..."
    },
    {
      "document_id": "my-doc-001",
      "document_name": "Annual Report",
      "chunk_index": 7,
      "score": 0.712,
      "image_base64": "R0lGODlhAQ..."
    }
  ],
  "total_results": 2
}
```

3.2.3 Chunk Object Fields

Field	Type	Description
document_id	string	The ID of the document this chunk belongs to
document_name	string	Human-readable document name
chunk_index	integer	Page number (1-indexed) within the document
score	float	Visual similarity score from ColQwen2 (higher = more relevant)
image_base64	string	Base64-encoded PNG image of the page

Table 6: Chunk metadata fields

3.2.4 Error Responses

Status Code	Description
404 Not Found	The specified document_id filter does not match any uploaded document.
422 Validation Error	Missing required query parameter, or top_k out of range.

Table 7: Get chunks error responses

3.2.5 Examples

cURL — Basic search:

```
curl "http://localhost:8010/api/documents/chunks?query=total+revenue"
```

cURL — Search with top_k:

```
curl "http://localhost:8010/api/documents/chunks?query=summary+of+
findings&top_k=3"
```

cURL — Filter by document:

```
curl "http://localhost:8010/api/documents/chunks?query=payment+details
&document_id=my-doc-001&top_k=2"
```

cURL — Pretty-print the response:

```
curl -s "http://localhost:8010/api/documents/chunks?query=revenue&
top_k=3" \
| python3 -m json.tool
```

3.3 Delete Document

Method	DELETE
Path	/api/documents/{document_id}
Content-Type	application/json (response)

Delete a document and all of its associated chunks. The document will no longer appear in search results.

3.3.1 Path Parameters

Parameter	Type	Required	Description
document_id	string	Yes	The unique ID of the document to delete

Table 8: Delete path parameters

3.3.2 Success Response — 200 OK

```
{
  "document_id": "my-doc-001",
  "message": "Document 'Annual Report' and all its chunks have been
             deleted."
}
```

3.3.3 Error Responses

Status Code	Description
404 Not Found	No document found with the given document_id.

Table 9: Delete error responses

3.3.4 What Happens on Delete

1. The Byaldi search index is rebuilt from scratch without the deleted document.
 2. The document's stored file is permanently removed from disk.
 3. The document is removed from the metadata registry.
- Note:** Deletion may take a few seconds for large indexes, as the entire index must be rebuilt.

3.3.5 Examples

cURL — Delete a document:

```
curl -X DELETE http://localhost:8010/api/documents/my-doc-001
```

cURL — Delete and pretty-print response:

```
curl -s -X DELETE http://localhost:8010/api/documents/my-doc-001 \
  | python3 -m json.tool
```


3.4 Health Check

Method	GET
Path	/health

Basic health check endpoint to verify the API is running.

3.4.1 Success Response — 200 OK

```
{  
  "status": "ok"  
}
```

3.4.2 Example

```
curl http://localhost:8010/health
```

4 Complete Workflow Example

This section demonstrates a full end-to-end workflow using `curl` in the terminal.

4.1 Terminal (cURL)

```
# Step 1: Check API health
curl http://localhost:8010/health

# Step 2: Upload a document
curl -X POST http://localhost:8010/api/documents/upload \
  -F "file=@report.pdf" \
  -F "document_id=report-2025" \
  -F "document_name=Annual Report 2025"

# Step 3: Search for relevant chunks
curl -s "http://localhost:8010/api/documents/chunks?query=total+
  revenue&top_k=3" \
  | python3 -m json.tool

# Step 4: Search within a specific document
curl -s "http://localhost:8010/api/documents/chunks?query=expenses&
  document_id=report-2025&top_k=2" \
  | python3 -m json.tool

# Step 5: Delete the document when done
curl -X DELETE http://localhost:8010/api/documents/report-2025
```

4.2 Shell Script (Automated Workflow)

```
#!/bin/bash
# Automated upload, search, and cleanup

BASE_URL="http://localhost:8010"

# Upload and capture document_id from response
DOC_ID=$(curl -s -X POST "$BASE_URL/api/documents/upload" \
  -F "file=@report.pdf" \
  | python3 -c "import sys,json; print(json.load(sys.stdin)['
    document_id'])")

echo "Uploaded document: $DOC_ID"

# Search for chunks
echo "Searching for relevant pages..."
curl -s "$BASE_URL/api/documents/chunks?query=revenue&top_k=3" \
  | python3 -m json.tool

# Delete when done
curl -s -X DELETE "$BASE_URL/api/documents/$DOC_ID" \
  | python3 -m json.tool
```

5 Frontend Integration

5.1 JavaScript (Fetch API)

```
const BASE_URL = "http://localhost:8010";

// --- Upload a document ---
const formData = new FormData();
formData.append("file", fileInput.files[0]);
formData.append("document_name", "My Report");

const uploadRes = await fetch(`${BASE_URL}/api/documents/upload`, {
  method: "POST",
  body: formData,
});
const { document_id, chunk_count } = await uploadRes.json();
console.log(`Uploaded: ${document_id}, ${chunk_count} chunks`);

// --- Search for chunks ---
const query = encodeURIComponent("total revenue");
const searchRes = await fetch(
  `${BASE_URL}/api/documents/chunks?query=${query}&top_k=5`
);
const { chunks } = await searchRes.json();

// --- Display a chunk's page image ---
const img = document.createElement("img");
img.src = `data:image/png;base64,${chunks[0].image_base64}`;
document.body.appendChild(img);

// --- Delete a document ---
await fetch(`${BASE_URL}/api/documents/${document_id}`, {
  method: "DELETE",
});
```

5.2 Python (requests)

```
import requests
import base64

BASE_URL = "http://localhost:8010"

# --- Upload ---
with open("report.pdf", "rb") as f:
    resp = requests.post(
        f"{BASE_URL}/api/documents/upload",
        files={"file": f},
        data={"document_name": "Annual Report"},
    )
doc_id = resp.json()["document_id"]
print(f"Uploaded: {doc_id}")

# --- Search ---
resp = requests.get(
    f"{BASE_URL}/api/documents/chunks",
```

```
    params={"query": "total revenue", "top_k": 3},
)
chunks = resp.json()["chunks"]

for chunk in chunks:
    print(f"    Page {chunk['chunk_index']} "
          f"(score: {chunk['score']:.3f}) "
          f"from {chunk['document_name']}")

# --- Save a chunk image to disk ---
if chunks:
    img_data = base64.b64decode(chunks[0]["image_base64"])
    with open("top_result.png", "wb") as f:
        f.write(img_data)

# --- Delete ---
resp = requests.delete(f"{BASE_URL}/api/documents/{doc_id}")
print(resp.json()["message"])
```

6 Configuration

All API settings are configured via environment variables in `Docker/.env`.

Variable	Default	Description
API_PORT	8010	Port the API server listens on
API_HOST	0.0.0.0	Host address to bind to
SEARCH_TOP_K	10	Default maximum search results
BYALDI_MODEL	vidore/colqwen2-v0.1	Visual retrieval model
IMAGE_DPI	200	DPI for PDF page rendering

Table 10: API-related environment variables

6.1 Running Locally (Without Docker)

```
# From the project root
cd src
PYTHONPATH=. python -m uvicorn api.server:app --host 0.0.0.0 --port 8010
```

6.2 Running with Docker

The API starts automatically as part of the Docker stack:

```
cd Docker
./start.sh          # Starts vLLM + Docker stack (includes API)
./start.sh --build  # Force rebuild if code changed
./start.sh --stop   # Stop all services
```

7 Response Schema Reference

7.1 UploadResponse

```
{
  "document_id": "string",
  "document_name": "string",
  "chunk_count": 0,
  "message": "string"
}
```

7.2 ChunksResponse

```
{
  "query": "string",
  "chunks": [
    {
      "document_id": "string",
      "document_name": "string",
      "chunk_index": 0,
```

```
    "score": 0.0,
    "image_base64": "string"
  },
  "total_results": 0
}
```

7.3 DeleteResponse

```
{
  "document_id": "string",
  "message": "string"
}
```

7.4 Error Response (HTTPException)

```
{
  "detail": "string"
}
```

8 Notes

- The API uses a **separate index** (`api_documents`) from the Chainlit chat UI (`documents`). Documents uploaded via the API do not appear in the Chainlit chat, and vice versa.
- **Chunk = Page.** Each page of a document is treated as one chunk. For single-page files (images), the document has exactly one chunk.
- **Image encoding.** The `image_base64` field contains a PNG-encoded image of the page. To display it in HTML: ``
- **Delete rebuilds the index.** Because the underlying Byaldi engine does not support removing individual documents, a delete operation rebuilds the entire index from the remaining documents. This is fast for small collections but may take longer for very large ones.
- **Interactive API docs.** Once the server is running, visit `http://localhost:8010/docs` for Swagger UI, or `http://localhost:8010/redoc` for ReDoc — both allow you to try endpoints directly in the browser.
- Replace `localhost` with your server's IP address when accessing the API remotely (e.g., from a RunPod instance or a different machine on the network).